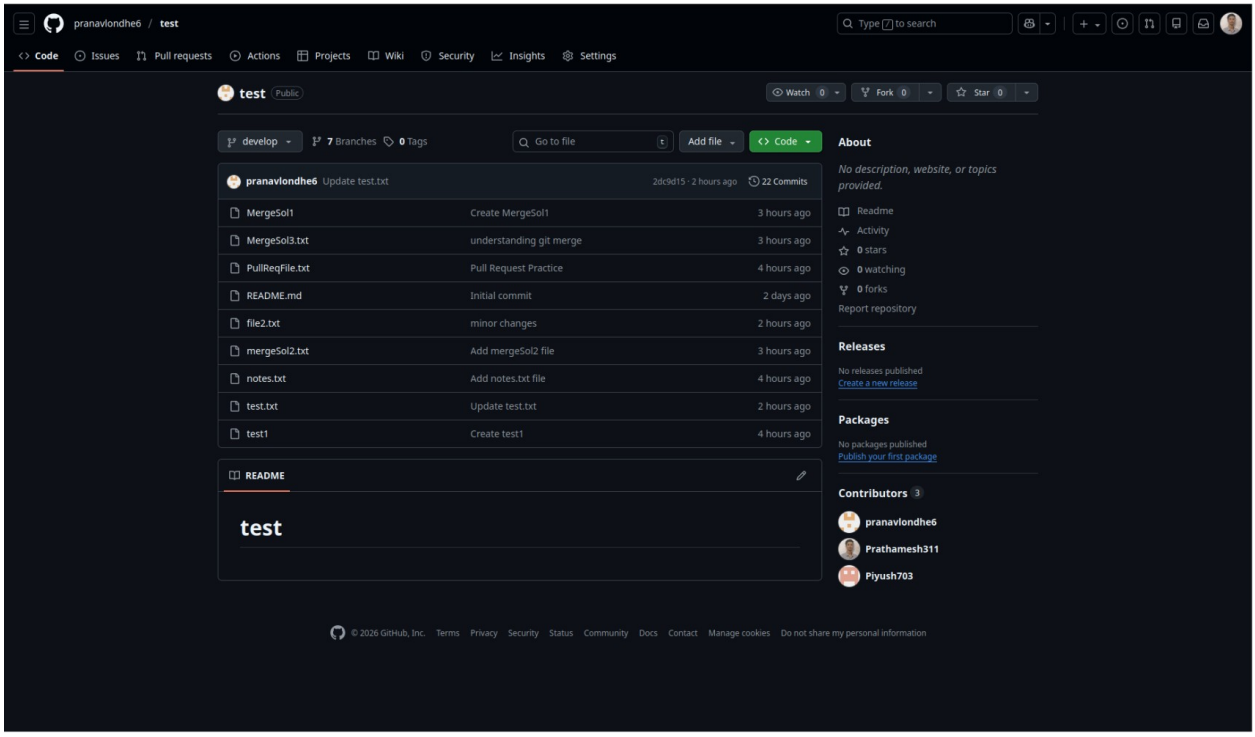
GIT ASSIGNMENT -1

Name : Prathamesh Shingate
Partner : Piyush and Pranav

---

1. Github repo created by Pranav and added the collaborators as Prathamesh311 and Piyush703 . so we can collaborate the repo called test.



1. Git Clone :

Creates a copy of a remote repository on your local machine.

Command – git clone repository URL

eg → **git clone git@github.com:pranavlondhe6/test.git**

```
prathamesh@Prathamesh-Shingate:~/Desktop$ git clone git@github.com:pranavlondhe6/test.git
Cloning into 'test'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
```

Now go to the respective cloned repository
eg → cd test

```
prathamesh@Prathamesh-Shingate:~/Desktop$ cd test
prathamesh@Prathamesh-Shingate:~/Desktop/test$ ls
README.md
```

2. Git fetch :

git fetch is used to check and download new updates from the remote repository.

Command – **git fetch**

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git fetch
From github.com:pranavlondhe6/test
 * [new branch]      feature2   -> origin/feature2
```

3. Git branch :

Listing branches: Shows all branches and highlights the current one.

1. Command : **git branch**

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git branch
* develop
```

2.Command : **git branch -a**

git branch -a shows every branch that exists locally and on the remote.

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git branch -a
* develop
  remotes/origin/HEAD -> origin/develop
  remotes/origin/develop
  remotes/origin/feature/test
  remotes/origin/feature2
  remotes/origin/master
```

4. Git checkout :

git checkout is used to switch between branches or restore files in a Git repository.

Command : **git checkout <branch-name>**

eg → git checkout feature2

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git checkout feature2
branch 'feature2' set up to track 'origin/feature2'.
Switched to a new branch 'feature2'
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git branch
  develop
* feature2
```

5. Git add :

Adds changes to the staging area in preparation for a commit**.**

Command : **git add <file-name>**

eg → git add file2.txt

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git add file2.txt
```

6. Git status :

Shows the status of changes as untracked, modified, or staged.

Command : git status

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git status
On branch feature2
Your branch is ahead of 'origin/feature2' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   file2.txt
```

7. Git restore :

Command **: git restore - -staged <file-name>**

Basically, it **unstages a file** that you previously added with git add <file-name>

eg → git restore - -staged file2.txt

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git restore --staged file2.txt
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git status
On branch feature2
Your branch is ahead of 'origin/feature2' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
```

8. Git commit :

git commit is used to save changes in your local repository. It takes the changes that have been staged (added with git add) and records them as a new commit with a message describing the changes.

Command : **git commit -m “<any-meassage> “**

eg → git commit -m “added file3.txt”

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git commit -m "added file3.txt"
[feature2 13a5790] added file3.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file3.txt
```

9. Git push :

Sends local changes to the remote repository.

Command : git push origin <branch-name>

eg → git push origin feature2

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git push origin feature2
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 237 bytes | 237.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:pranavlondhe6/test.git
   a1c7652..13a5790  feature2 -> feature2
```

10. Git pull :

Fetches changes from a remote repository and merges them into the current branch.

Command : git pull

eg → git pull origin feature2

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git pull origin feature2
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 902 bytes | 902.00 KiB/s, done.
From github.com:pranavlondhe6/test
 * branch            feature2   -> FETCH_HEAD
   13a5790..4516e96  feature2   -> origin/feature2
Updating 13a5790..4516e96
Fast-forward
 new-1 | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 new-1
```

11. git merge :

Combines changes from the specified branch into the current branch.

Command : **git merge <branch-name>**

eg → git merge feature4

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git merge feature4
Merge made by the 'ort' strategy.
 new1     | 1 +
 newfile2 | 1 +
 story1   | 0
 story2   | 1 +
 4 files changed, 3 insertions(+)
 create mode 100644 new1
 create mode 100644 newfile2
 create mode 100644 story1
 create mode 100644 story2
```

12. git stash :

used to **temporarily save (or “stash”) your uncommitted changes** so you can work on something else without committing them.

Command : **git stash** → **saves changes**

git stash apply → **apply stashed changes back**

example :

created a file story1.txt and save it , And added it to staging it area.

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ ls
README.md  story1.txt
prathamesh@Prathamesh-Shingate:~/Desktop/test$ cat story1.txt
this is the stashed file prathamesh@Prathamesh-Shingate:~/Desktop/test$ git status
On branch feature5
Your branch is up to date with 'origin/feature5'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .idea/
        story1.txt

nothing added to commit but untracked files present (use "git add" to track)
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git add story1.txt
```

after that stash it.

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git stash
Saved working directory and index state WIP on feature5: c2e3372 Initial commit
```

Then created a another file called story2.txt worked on it , and then commited it.

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ ls
README.md  story2.txt
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git add story2.txt
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git commit -m "added story2.txt"
[feature5 39953b3] added story2.txt
 1 file changed, 1 insertion(+)
 create mode 100644 story2.txt
```

Then apply stashed changes back .

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git stash apply
On branch feature5
Your branch is ahead of 'origin/feature5' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   story1.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .idea/

prathamesh@Prathamesh-Shingate:~/Desktop/test$ ls
README.md  story1.txt  story2.txt
```

So I used stash to save my uncommitted changes temporarily so I can work on another files .

13. git revert :

undo a specific commit by creating a new commit that reverses its changes, without removing the original commit from history.

Command : **git revert <commit-id>**

example :

Created a file abc.txt through terminal and added content in the file like “this is the sample file for revert” and commited it .

```
Terminal   Local (2)

prathamesh@Prathamesh-Shingate:~/Desktop/test$ echo "this is the sample file for revert" > abc.txt

prathamesh@Prathamesh-Shingate:~/Desktop/test$ git add abc.txt

prathamesh@Prathamesh-Shingate:~/Desktop/test$ git commit -m "abc.txt added"
[feature5 9d78207] abc.txt added
 2 files changed, 2 insertions(+)
 create mode 100644 abc.txt
 create mode 100644 story1.txt
```

Now again the content of abc.txt file is updated .

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ echo "this is the revert demo " >> abc.txt

prathamesh@Prathamesh-Shingate:~/Desktop/test$ git add abc.txt

prathamesh@Prathamesh-Shingate:~/Desktop/test$ git commit -m "updated the file"
[feature5 0834a27] updated the file
 1 file changed, 1 insertion(+)
```

So we have the command called git log - -oneline , it will show the commit history in single line.

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git log --oneline
0834a27 (HEAD -> feature5) updated the file
9d78207 abc.txt added
39953b3 added story2.txt
c2e3372 (origin/master, origin/feature5) Initial commit
```

Now I want to revert a latest commit.

Eg → git revert 0834a27

```
prathamesh@Prathamesh-Shingate:~/Desktop/test$ git revert 0834a27
[feature5 faf0b64] Revert "updated the file"
 1 file changed, 1 deletion(-)

prathamesh@Prathamesh-Shingate:~/Desktop/test$ cat abc.txt
this is the sample file for revert

prathamesh@Prathamesh-Shingate:~/Desktop/test$ git log --oneline
faf0b64 (HEAD -> feature5) Revert "updated the file"
0834a27 updated the file
9d78207 abc.txt added
39953b3 added story2.txt
c2e3372 (origin/master, origin/feature5) Initial commit
```

So , the latest commit is reverted .

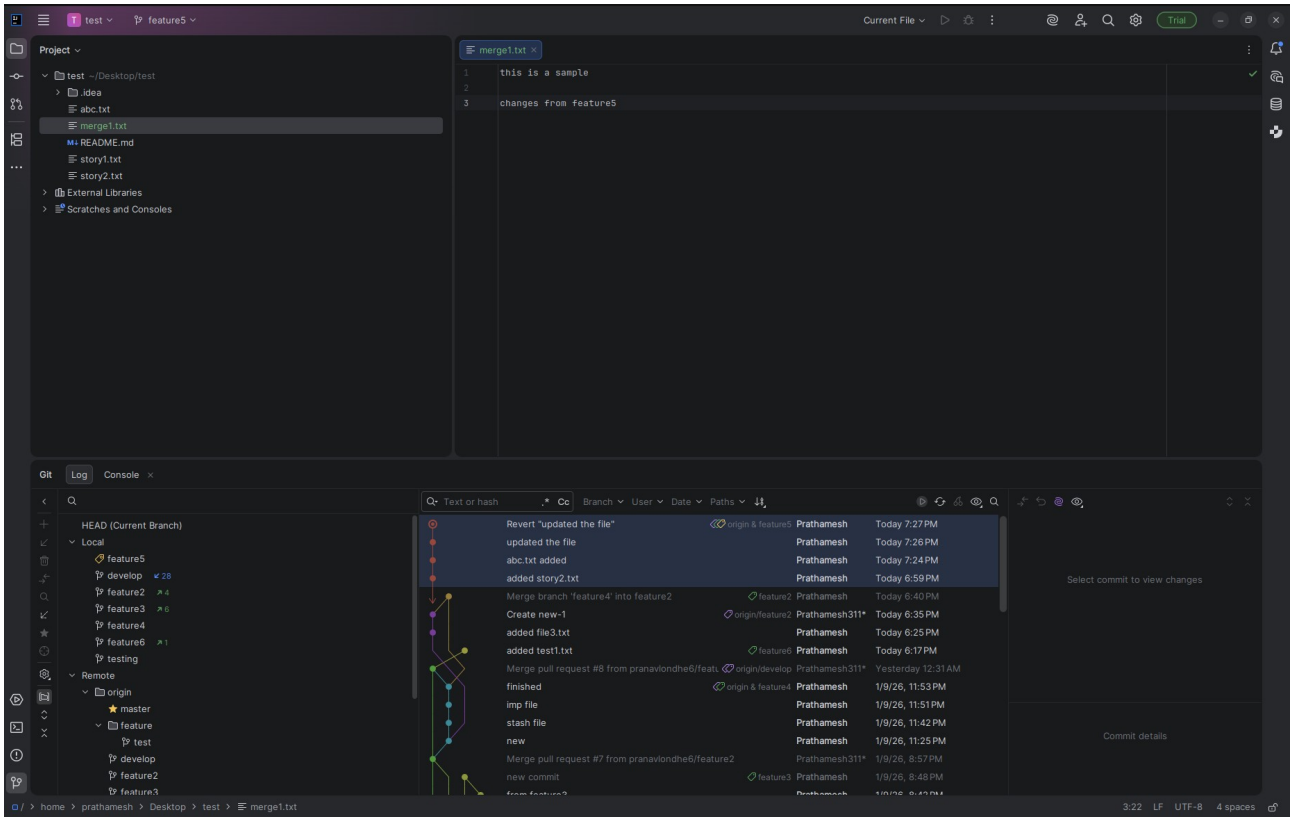
14. Git Merge Conflict :

A situation in Git where automatic merging fails because two or more branches have conflicting changes in the same file, and the user must manually resolve the differences before completing the merge.

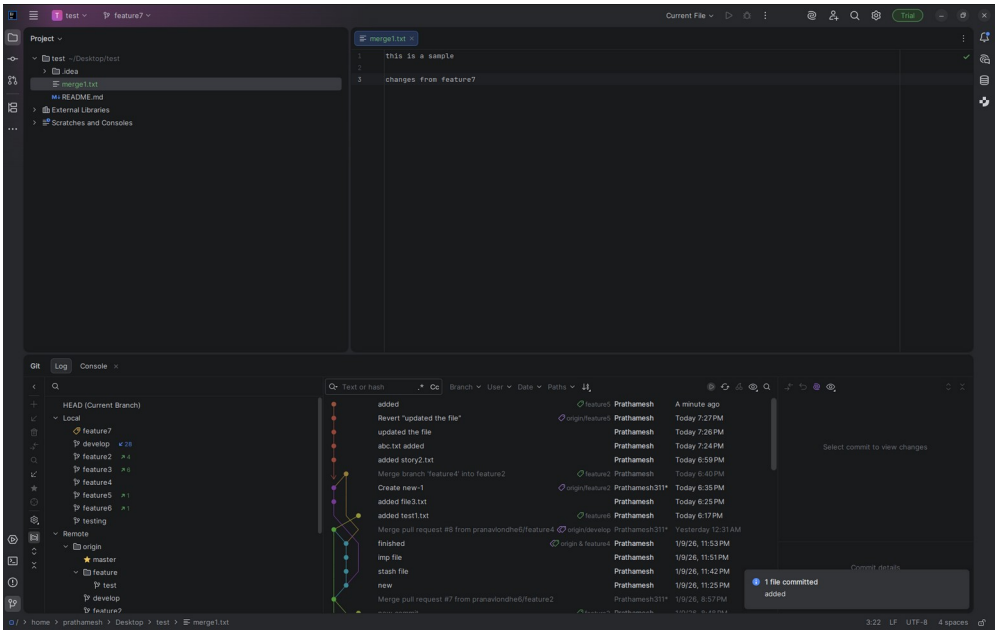
Example :

I created a merge conflict by creating a file with the same name (merge1.txt) on two different branches, each containing different content, and then resolved the conflict manually using the IDE.

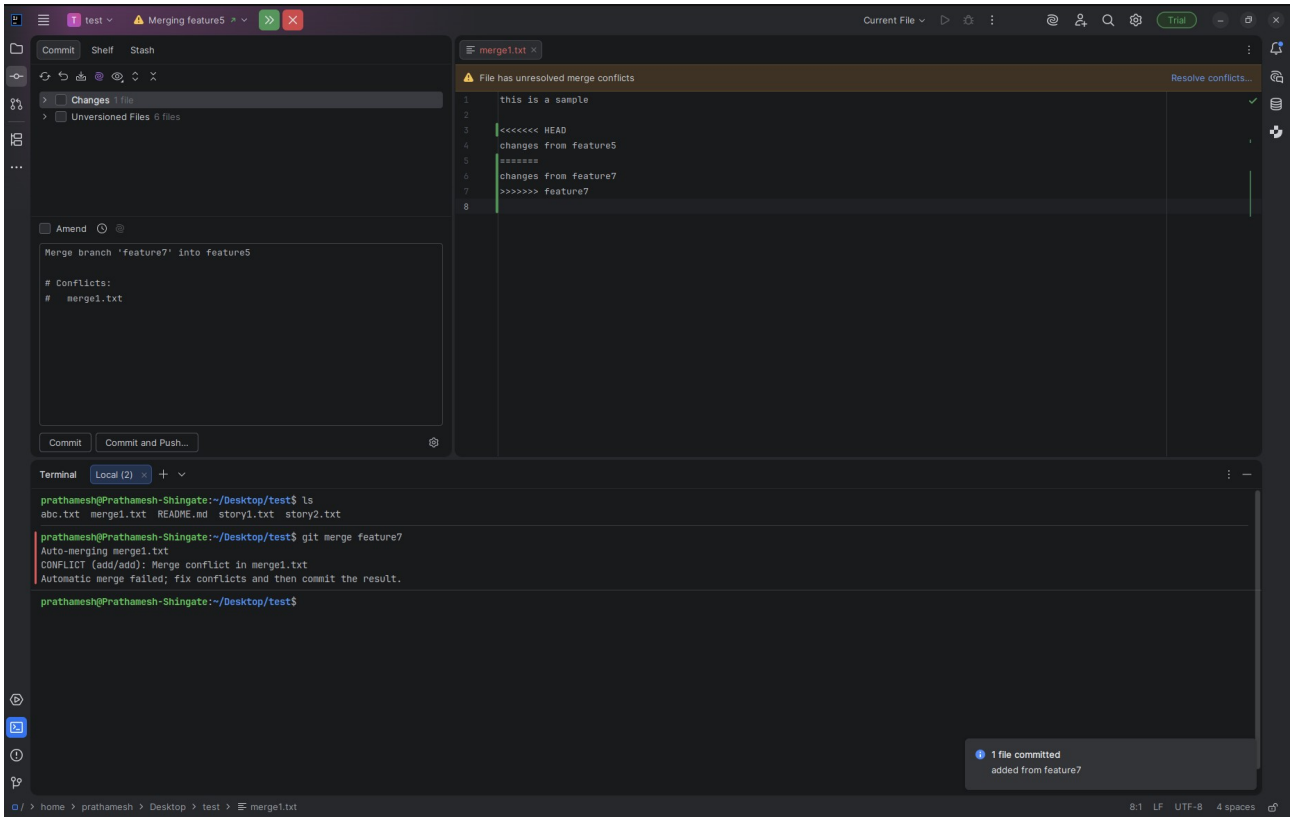
This is the file on feature5 called merge1.txt and commited it.

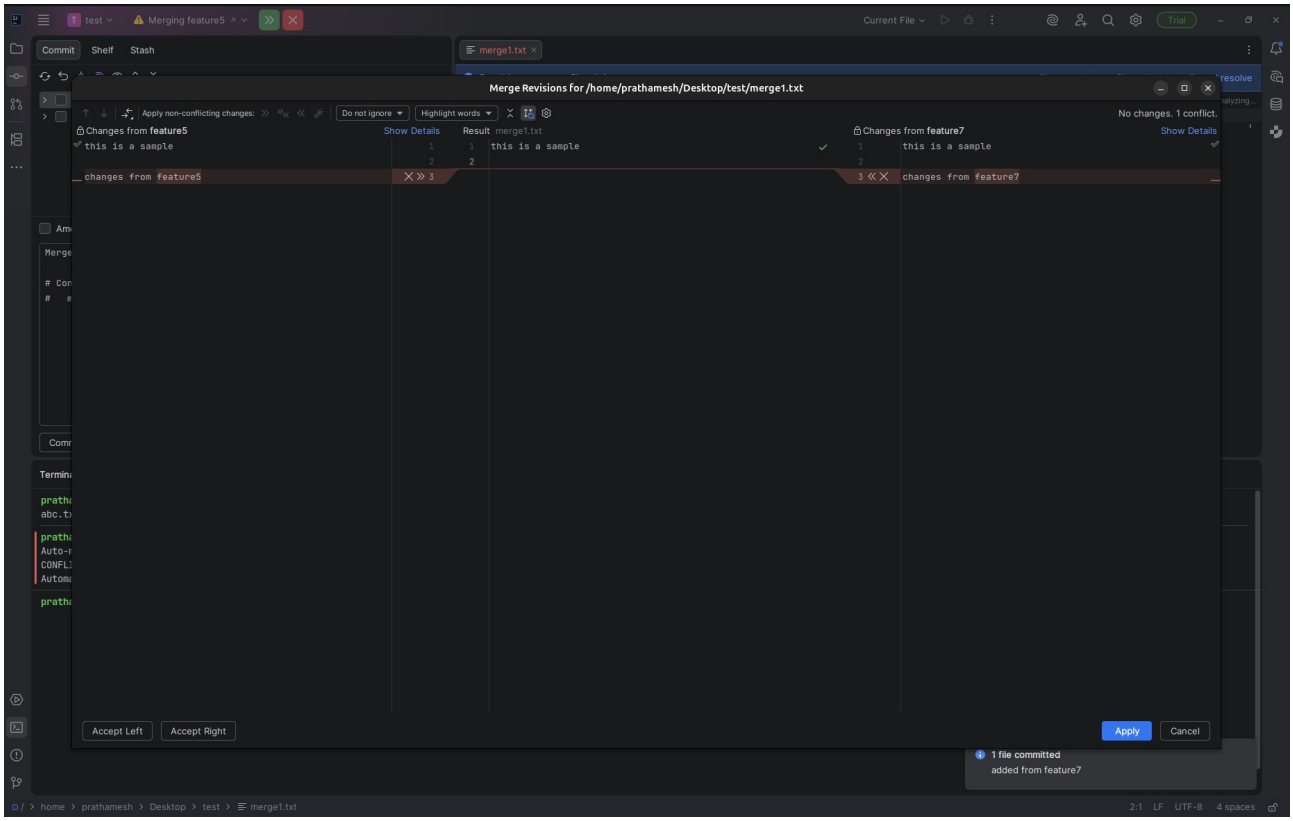


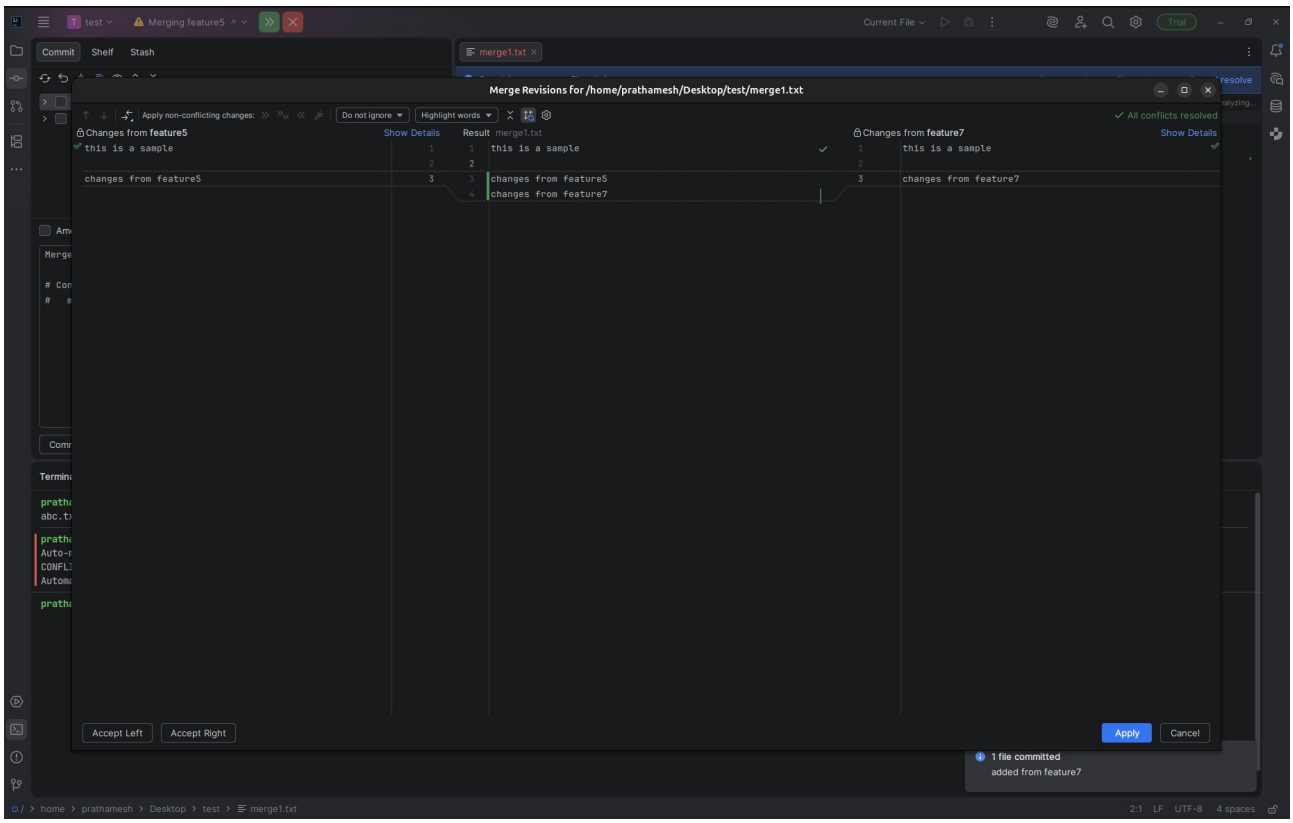
On branch feature7 I have the same file merge1.txt with different content on same line. And commited it .



When I merged feature5 branch with feature7 branch the conflict occurs







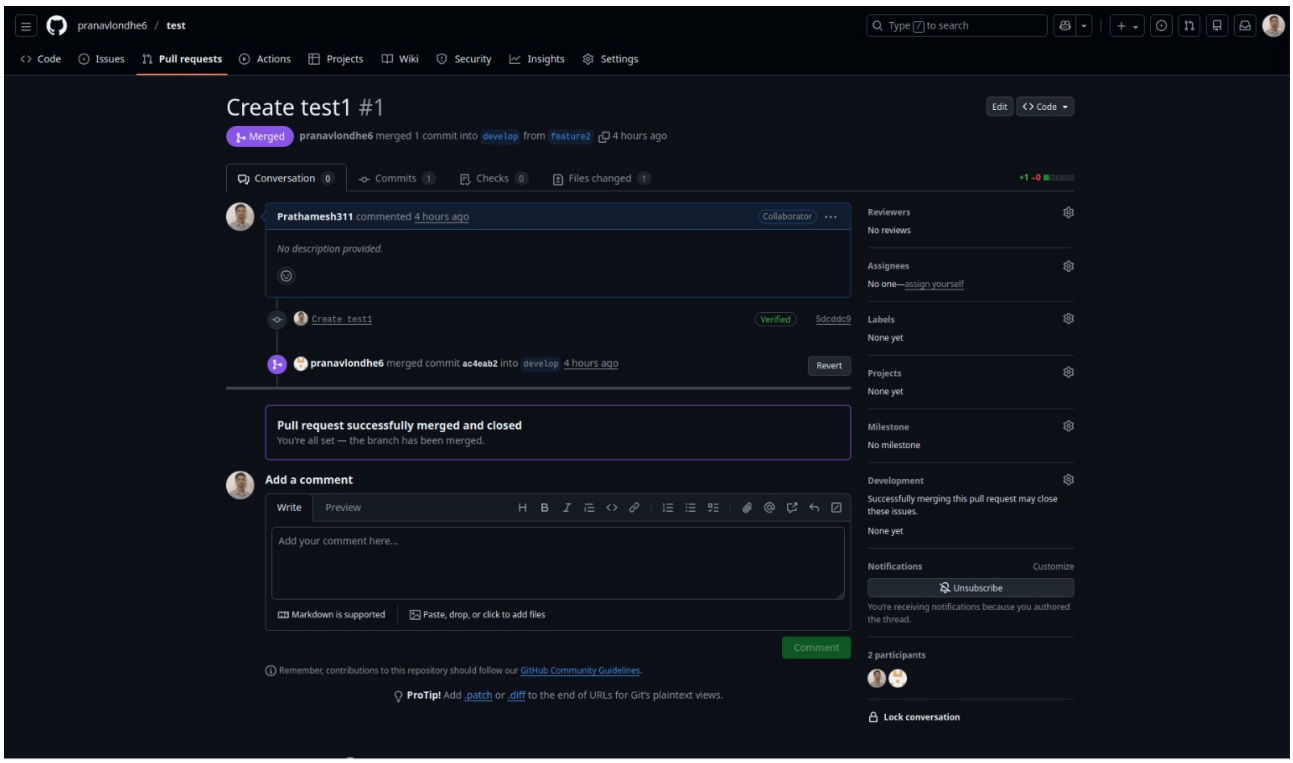
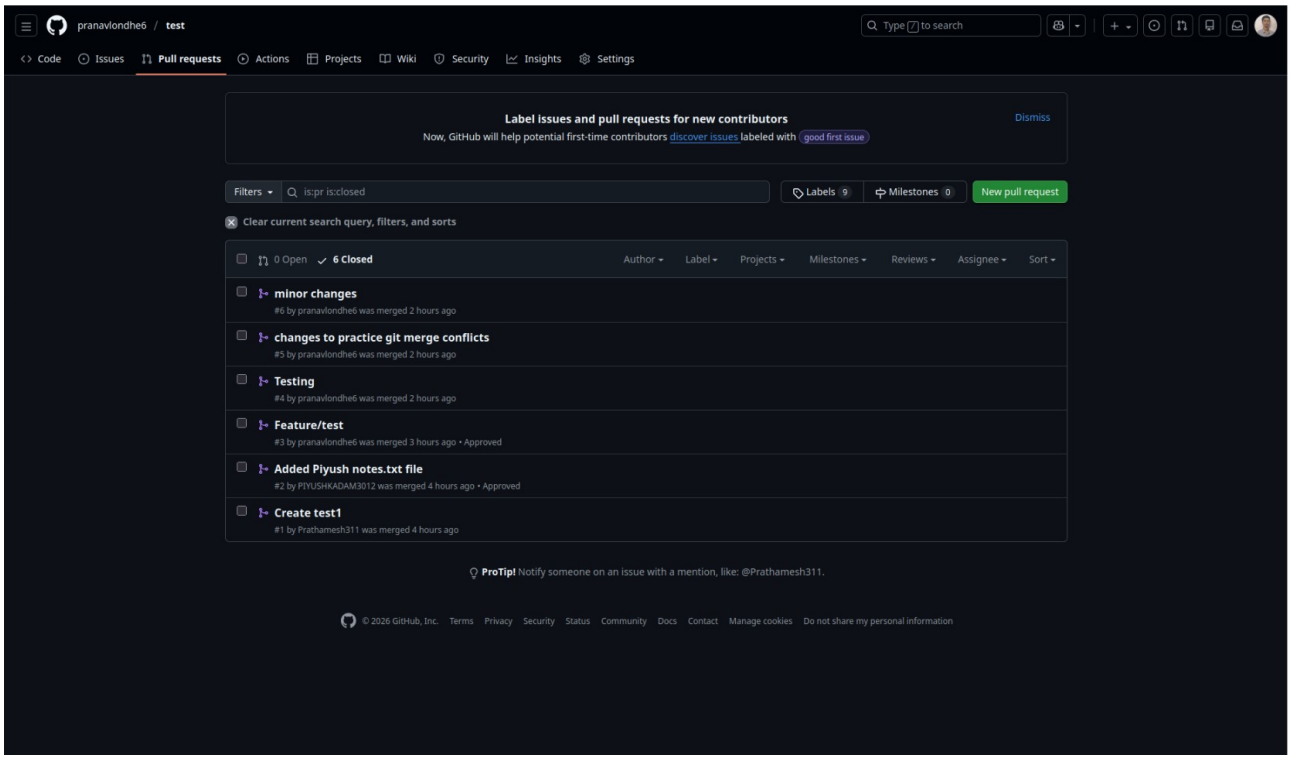
by keeping the changes of both. I successfully merge feature5 branch to feature7.

15. Pull Request :

A request to merge changes from one branch into another, allowing team members to review, discuss, and approve the code before it is merged.

Example :

Created a feature2 branch from develop branch and added some files and raise the pull request , that are approved and checked by pranav and piyush and then merged to develop branch.



So as a team we created our own features branches from develop and created PR and closed them.